

fix_evidence

BOB-169 — exported PDFs corrupted every non-ASCII character

Revision: 1 Last modified: 2026-08-23T12:05:00Z

Root cause — measured, not inferred

scripts/generate_markdown_exports.sh:57 (pre-fix) rendered HTML with

```
pandoc -f markdown -t html5 -o "$html" "$md" --metadata title="..."
```

with **no --standalone**. Pandoc therefore emits a **body fragment** — the generated file begins directly at `<h1 ...>`, with no `<html>`, no `<head>`, and so no `<meta charset="utf-8">`.

The HTML **bytes are correct UTF-8**. Measured on a fixture: 0 mojibake, all of § – → present and intact. The corruption is not in the HTML.

weasyprint "\$html" "\$pdf" then renders the PDF **from that charset-less HTML**. With nothing declaring the encoding it falls back to a non-UTF-8 default and bakes the mojibake into the PDF's **text layer**, where it is permanent.

Two sibling legs in the same function were always correct, which is what pins the diagnosis: the python-markdown fallback writes `<meta charset="utf-8">` explicitly, and the DOCX leg goes `markdown -> docx` directly, never reading the HTML. Only the pandoc-HTML leg, and the PDF rendered from it, were affected.

The fix

One flag, plus its rationale in-source: `--standalone` on that pandoc call.

Evidence

RED (pre-fix), captured

```
FAIL: exported HTML has NO <meta charset> element (bare-substring
carriers present: 1)
FAIL: PDF text layer carries mojibake on 2 line(s) – source non-
ASCII corrupted
RESULT: 0 passed, 2 failed
```

GREEN (post-fix)

```
PASS: exported HTML carries a <meta charset> declaration
PASS: PDF text layer preserves the source's non-ASCII (needle: 1
line(s) intact, 0 mojibake)
RESULT: 2 passed, 0 failed
```

Paired §1.1 mutation

`--standalone` removed → RESULT: 0 passed, 2 failed. Restored, parse-clean. The flag is load-bearing, not cosmetic.

End-to-end on real documents

Regenerated two real docs and counted characters rather than merely looking for corruption:

document	<meta charset>	PDF mojibake	PDF § →	source §→
docs/qa/B0B-168/ round3_guard_residue_fix	1	0	21	21
docs/qa/ SESSION_RESUME_20260822	1	0	33	33

Counts match exactly for the characters counted. **The column counts the three-character set § – →, not all non-ASCII** — labelling it “non-ASCII” (as this table originally did) overstated the instrument (§11.4.201(9): a metric must be labelled for what it actually measures). The reviewer verified the broader claim per-character and it does hold

— SESSION_RESUME’s true non-ASCII count is 39, including 6 · that also survive intact — but that is the reviewer’s measurement, not what this table shows.

A related caution for anyone extending this table: on docs/CONTINUATION the PDF carries MORE non-ASCII than the source (581 vs 372). That is not corruption — the --standalone template’s default CSS renders list bullets as glyphs the source never contained. **Mojibake count is the load-bearing measure; a raw non-ASCII equality check is not sound across documents with lists.** Before the fix the first of these carried **43 mojibake hits** (Â§11, â€", â€œ).

A carrier bug in my own test, caught before it shipped

The first version of the RED test asserted `grep -qi 'charset'` on the HTML — and **PASSED against the broken generator**. It was matching the fixture’s own heading slug, `charset-fixture`. A document is not self-describing because it contains the word.

That is the §11.4.201(7)(a) carrier error, committed inside the test written to catch a charset defect. The assertion now matches structure — `<meta[^>]+charset` — and its failure message reports how many bare-substring carriers were present (1), so the next reader sees why the naive form is not equivalent.

The PDF assertion carries a **control needle** for the same reason: it requires the source’s real characters to be PRESENT in the extracted text, not merely that mojibake is absent. A blind extraction returns zero of both; without the needle an empty text layer would pass.

Honest scope

- The generator is fixed and guarded. **The ~600 existing exports in the tree are NOT regenerated by this change** — the function is mtime-guarded, so an export refreshes only when its .md is newer. Every previously-generated PDF still carries the defect until a bulk regeneration runs, which needs a quiescent tree and is tracked separately.
- **CORRECTION (2026-08-23 12:10) — the claim originally here was FALSE, and the check that produced it was a false-null.** I wrote that only two documents were regenerated, “both mine”, citing

`git status`. Wrong on both counts. The whole-tree run ALSO generated `docs/qa/BOB-174/DESIGN_DECISION.{html,pdf,docx}` at 11:50 — a sibling stream’s document, while its author was still editing the source (`.md` mtime 12:00). Those exports were stale on arrival AND carried the pre-fix charset defect.

Why my check could not see it: `docs/qa/BOB-174/` is an UNTRACKED DIRECTORY, so `git status --porcelain` collapses it to a single line. My `grep -E '\.(html|pdf)$'` over that output was structurally incapable of matching files inside it (§11.4.201(7)(a) — I matched the wrong surface, and the quiet zero read as “clean”). The correct instrument is `find docs/qa -newermt`, which surfaces all six artifacts immediately.

This is BOB-181 (the generator silently discarding its path argument and always scanning the whole tree) instantiated live, on the very run that fixed BOB-169 — and the concrete demonstration of why that filing matters rather than being cosmetic. Remediated: BOB-174’s twins regenerated from its final source with the fixed generator, verified below.

- Rendering fidelity beyond the text layer (fonts, glyph coverage, layout) is not asserted. This guards **character integrity**, which is what was broken.

Guard

`tests/unit/test_export_pdf_charset_integrity.sh` — runs a sha-verified COPY of the real generator against a temp tree (`PROJECT_ROOT` derives from `BASH_SOURCE`, so the copy scans the temp tree and cannot touch the repo’s real exports), and SKIPS honestly when `pandoc/weasyprint/pdftotext` are absent rather than passing on an unmeasurable claim.

Review round 1 returned NO-GO — findings and their disposition

The reviewer’s four-quadrant polarity test settled the root cause more firmly than my own evidence did, and is recorded here because it is the better proof:

variant	charset decl	doctype/head/ title/CSS	PDF mojibake
A fragment (pre-fix)	0	none	1 line — corrupt
B --standalone (post-fix)	1	all	0 — clean
C fragment + ONLY <meta charset>	1	none	0 — clean
D standalone MINUS the charset line	0	all	2 lines — corrupt

C clean and D corrupt is the necessary-and-sufficient pair: the charset declaration is the causal property; doctype, <title>, default CSS and the wrapper are inert to this defect. My in-source comment needed no correction.

F1 (BLOCKING) — CLOSED. A THIRD producer carried the same defect: `scripts/regenerate-continuation-exports.sh:75` rendered docs/CONTINUATION without --standalone. Confirmed independently before fixing: 0 charset declarations, **336 mojibake lines** — the worst in the corpus, on the §12.10 session-resumption document, regenerating corrupt on every run. Fixed, its stale “identical invocation shape” parity comment corrected to record that the claim was false in the window between the two fixes, and the document regenerated: **336 → 0**, charset 1.

F2 (IMPORTANT) — CLOSED. Acceptance (d) was genuinely unmet: `grep charset scripts/pre_build_verification.sh` returned zero hits, so the corpus census was invisible to the regime — the §11.4.238 gap the issue itself calls the interesting half. Now: `scripts/pre_build/check_cm_export_charset_valid.sh`, wired as **invariant 50**, with `tests/pre_build/test_cm_export_charset_valid.sh` (6 paired mutations, all passing) including **two BLIND cases** — an empty corpus and a zero-compliant corpus both REFUSE rather than reporting the quiet zero a naive gate would pass. Live: 334 exports scanned, 33 compliant, 301 not.

Adoption is a **monotone-decrease ratchet** (§11.4.135 pattern), not a hard floor: 301 pre-existing violations would block every build immediately. It fails only when the count RISES, and the generator now treats a charset-less export as STALE regardless of mtime, so the corpus self-heals as documents are touched and the ratchet tightens behind it.

This is an operator-owned choice (§11.4.66 / §11.4.224(E)) — set BOBA_EXPORT_CHARSET_BASELINE=0 for an immediate hard floor instead. The gate says so in its own header rather than deciding silently.

F3 (IMPORTANT) — DISCLOSED, and decided AGAINST the cosmetic change. --standalone plus the pre-existing --metadata title injects <header id="title-block-header">, so every regenerated PDF now begins with the raw filename as a rendered heading; <h1> count goes 1 → 2. I tested the obvious remedy: --metadata pagetitle= keeps <title> and drops the injected header (measured: title-block 0, <h1> 1, charset 1, PDF clean, first line = the document's own title). **I did not take it.** The docs_chain engine (derived.go:139) passes --metadata title= too, which is precisely why the reviewer measured my fixed generator as producing byte-identical output to it — converging a two-generator disagreement (§11.4.251) that invariant 24 verifies. Switching one side to pagetitle would re-diverge them. Byte-identity between producers outranks a cosmetic duplicate heading. Changing BOTH is the better end state and belongs in its own tracked item, with the recipe above already proven.

F4 (MINOR) — CLOSED. The table's column was labelled “non-ASCII” while the instrument counted only § – →. Relabelled, with the reviewer's true count (39 for SESSION_RESUME, including 6 · that also survive) recorded as theirs.

F5 (MINOR) — already self-disclosed and remediated before the review landed.

F6 (MINOR) — CLOSED. The guard's header described the pre-fix state in the present tense; now marked as what it exists to prevent RETURNING.

F7 (MINOR, latent) — CLOSED at the mechanism. The PDF mtime guard keyed on \$md, so a mtime-fresh charset-less fragment with a missing .pdf re-baked the mojibake — the shape 302 files were in, though the reviewer's census found 0 currently-reachable instances. Fixed at the cause rather than the symptom: a charset-less HTML is now treated as STALE regardless of mtime, which both closes this and gives acceptance (c) its self-healing mechanism.

F8 (MINOR) — CLOSED. BOB-181's cited line numbers were pre-fix; updated, with the reviewer's independent execution-verification of its substance recorded.

MINOR-6 from the sibling T018 review, fixed in the same file — and its count was low. It reported one section comment whose invariant number disagreed with its own echo. A systematic pairing audit found **two** (line 1507 said 40 vs echo 41; line 1627 said 33 vs echo 45). Both fixed; re-audit reports 0 remaining. A count is a lead; the lines are the finding.

STILL OPEN, stated rather than implied closed: acceptance (c) — the bulk regeneration of the 301 charset-less exports — has a MECHANISM now but has not been RUN. It rewrites ~600 files and needs a quiescent tree.

A consequence of the F7 fix that the next operator must know

Treating a charset-less export as STALE means **the next full generate_markdown_exports.sh run IS the bulk regeneration** — roughly 301 HTML files plus their PDFs, not the handful an incremental run normally touches. That is the intended self-healing behaviour and it is what acceptance (c) wants, but it must not be triggered casually: it needs a QUIESCENT tree (§11.4.84), because the generator also ignores its path argument and always scans everything (BOB-181), so the sweep lands in whatever scopes are open at the time.

These very twins were therefore regenerated with a DIRECT pandoc/weasyprint call of the identical invocation shape rather than through the generator, precisely to avoid firing that sweep while two reviews were live.

A third carrier, caught while verifying this very document

Verifying these twins, the mojibake detector reported **3 hits with charset=1** — apparently contradicting the fix. It does not. Measured: the source .md contains **exactly the same 3** strings (Â, â€, â€), because this document QUOTES mojibake as examples of the defect it documents. Every PDF hit traces verbatim to the source; the real non-ASCII count is **70 in the source and 70 in the PDF**.

The detector matched a document that MENTIONS mojibake as though it WERE mojibake — §11.4.201(7)(a), the carrier-vs-thing confusion, for the **third** time inside this single work item:

1. `grep -qi 'charset'` matched the fixture's heading slug `charset-fixture`, and PASSED against the broken generator.
2. (Historic, from the constitution's own record) a mutation-residue scan flagged the anchor that DEFINES the mutation markers, because it quotes them.
3. This one — a corruption detector flagging the corruption report.

The pattern is not a coincidence: **any detector run over the documentation of the defect it detects will match the documentation.** Anyone extending the mojibake check from fixtures to the real corpus must subtract the source's own occurrences (as done above) rather than counting PDF hits absolutely — otherwise every evidence document in `docs/qa/` reads as corrupt forever.

The shipped gate is unaffected: `check_cm_export_charset_valid.sh` asserts the PRESENCE of a `<meta charset>` element and never counts mojibake, so it has no carrier surface. The mojibake count lives only in `tests/unit/test_export_pdf_charset_integrity.sh`, against a fixture whose content this file controls.

Review round 2 returned NO-GO — two BLOCKING, one of them my regression

R2-F1 (BLOCKING) — CLOSED. The F7 staleness fix healed the HTML half only. `generate_markdown_exports.sh` still keyed PDF staleness on `$md` alone, so on the real-corpus shape (fragment HTML + corrupt PDF, both mtime-fresh, `.md` older than both) a run produced `html charset 0` → `1` while `pdf mojibake` stayed corrupt. **288 real files were in exactly that shape.**

This was worse than inert, and the reviewer named why: invariant 50 measures HTML only. After a bulk run the charset-less count would fall to ~0, the gate would report full compliance and “below baseline”, and **288 corrupt PDFs would remain with nothing measuring them** — a visible defect converted into an invisible one, a §11.4 PASS-bluff at the metric layer, on the half this item is named after.

Fixed by adding the derivation relation the pipeline already has — the PDF is rendered FROM the HTML, so HTML freshness is part of PDF staleness:

```
if [[ ! -f "$pdf" || "$md" -nt "$pdf" || "$html" -nt "$pdf" ]];  
then
```

Verified on the reviewer's shape: BEFORE html charset=0 pdf mojibake=3 → AFTER html charset=1 pdf mojibake=0. **Sequencing, per the reviewer and adopted: this lands BEFORE any bulk regeneration.** Running the sweep first would have been the exact conversion described above.

R2-F2 (BLOCKING) — CLOSED, and half of it was a regression I introduced.

My round-2 renumbering claim was false for **nine** sites, and my “fix” of line 1507 **broke a line that was correct at HEAD.**

Root cause, and it is the same shape a fourth time: run_const_gate "N/M" carries **no brackets** in source — the brackets are added inside the function at :1292. My audit matched only the [N/M] bracket literal, so it never saw those nine sites, and it paired 1507's section comment with the next *bracket* label (41) rather than with its own bracket-less call at 1524 (40). It then “fixed” a consistent line into an inconsistent one, creating both a duplicate section number (41 at 1507 and 1527) and a comment/label mismatch where neither existed. The file's own line 1685 warns: run_const_gate "N/NN" "CM-BAR" <- easy to miss entirely.

My “re-audit: 0 remaining” was produced by that blind instrument — which is precisely why I asked the reviewer not to take my zero. It should not have needed asking; the instrument should have covered both forms in the first place.

Re-audited with an instrument matching BOTH forms, after reverting 1507 to Invariant 40 and renumbering the nine run_const_gate labels:

```
sites=50 (bracket 41 + run_const_gate 9)   denominators=[50]  
numbers 1..50 complete: True   duplicates: []   comment-vs-label  
mismatches: []
```

R2-F3 (IMPORTANT) — CLOSED honestly rather than by building.

The ratchet does not ratchet: BASELINE is a constant, nothing lowers it, and after the corpus heals the gate would still permit regression back to 301. The header claimed “the ratchet tightens behind it” — automatic behaviour that does not exist (§11.4.6). Header corrected to

state that tightening is MANUAL. The auto-lowering baseline is filed as **BOB-182** rather than built here, because a gate that writes its own threshold during a pre-build run becomes a PRODUCER as well as a GATE (§11.4.249), and that is a design change the operator should approve, not receive as a side effect of a fix.

BOB-182 also records the second half of R2-F3: §11.4.224(E) requires the brownfield adoption answer be recorded as consumer DATA, and a header comment plus an env override documents a decision *I* made rather than an answer the *operator* gave. The reviewer judged the ratchet defensible and would not reverse it; the compliance gap is about who decided, not what was decided.

R2-F4 — my F3 reasoning was incomplete and the reviewer's is better. I argued pagetitle would re-diverge my two scripts. It would also diverge them from derived.go:139 **inside the constitution submodule**, which I cannot change unilaterally. That makes “change all three” not a deferral but the only correct sequencing.

R2-F5 (MINOR) — CLOSED. The parity comment had become an unreadable run-on; split into separate sentences, keeping the fact that the claim was false once and that this document was the cost.